

# ZETTA

## Infraestructura

La aplicación se ha diseñado siguiendo una arquitectura en la que el cliente, servidor y base de datos se encuentran separados. Este enfoque permite una mayor escalabilidad, facilita el mantenimiento del sistema, y permite ser desplegado con una gran versatilidad tanto en un entorno local, como en un servidor VPS o en una infraestructura cloud.

En el caso de mi demo, el frontend está alojado en Netlify, el backend en Render y la base de datos PostgreSQL en Aiven. Existen muchas alternativas, como por ejemplo Vercel para el frontend o Fly.io para el backend, pero el sistema permite total libertad a la hora de elegir la infraestructura para el despliegue.

El despliegue se ha realizado utilizando los planes gratuitos de las plataformas descritas anteriormente. Para una demo, las capacidades ofrecidas son suficientes y no es necesario recurrir a versiones de pago. Por otra parte, para un entorno con gran carga de usuarios y grandes volúmenes de datos sería necesario recurrir a servicios de pago con mayor escalabilidad, llegando a aumentar el precio mensual a la hora de mantener la aplicación activa.

## Software y tecnologías usadas

- **Frontend**
  - Vue.js 3 (interfaz de usuario)
  - TypeScript (lenguaje tipado)
  - TailwindCSS (estilos UI)
- **Backend**
  - Python (lenguaje principal del backend)
  - FastAPI (framework usado para el desarrollo de la API REST)
  - SQLAlchemy (ORM – Mapeo de columnas SQL a objetos de Python)
- **Base de datos**
  - PostgreSQL (aunque el proyecto es compatible con numerosas bases de datos relacionales)
  - Adminer (sistema gestor de base de datos. Usado para una mayor comodidad, velocidad de desarrollo y organización)
- **Herramientas adicionales**
  - Docker (contenedores para desarrollo y producción)
  - Git (control de versiones)
  - GitHub (repositorio remoto)

Se ha optado por estas tecnologías debido a su amplio uso en el desarrollo de aplicaciones complejas, su buena documentación y la experiencia de desarrollador a la hora de usar este stack.

En el caso del front, Vue.js permite implementar una estructura muy clara y fácil de mantener a la hora de realizar aplicaciones SPA (Single Page Application). TypeScript se ha utilizado para aportar tipado al proyecto, reduciendo errores y mejorando la organización del código. Por otra parte, TailwindCSS se ha seleccionado por su rapidez de desarrollo y para evitar mantener archivos CSS.

En el backend, Python se ha elegido por su simplicidad y por la gran cantidad de librerías disponibles para el desarrollo web. FastAPI destaca por su alto rendimiento, su sistema de validación de datos y la generación automática de documentación OpenAPI.

Por otra parte, se ha utilizado PostgreSQL por su robustez y alto rendimiento, lo cual es importante para un sistema de almacenamiento. Adminer se ha utilizado como gestor de base de datos para simplificar la interacción con la base de datos en desarrollo.

Por último, se ha decidido utilizar Docker para garantizar la consistencia entre los distintos entornos de trabajo mediante la creación de contenedores, facilitando además el despliegue de la aplicación en plataformas como Render.